

AD-GAN: AXIS-DECOMPOSED GAN FOR SYMBOLIC PIANO MUSIC GENERATION

Shaunak Mohagaonkar

Mathematics and Computing Department
Manipal Institute of Technology, Manipal

ABSTRACT

I present **Axis-Decomposed GAN (AD-GAN)**, a novel generative adversarial architecture for symbolic piano music that explicitly separates temporal and harmonic processing into distinct convolutional branches. Unlike existing approaches such as MuseGAN and MidiNet that apply generic 2D convolutions over the joint time–pitch space, my model decomposes these two fundamental musical dimensions using separable Conv2d kernels, $(k \times 1)$ for temporal modelling and $(1 \times k)$ for pitch modelling, and fuses their outputs through learned 1×1 projections at every stage. The pitch branch further employs five parallel dilated convolutions at musically meaningful intervals (semitone, minor third, major third, perfect fifth, octave), such that the model is able to encode musically accurate tonal structure. The model is trained using the Wasserstein GAN with Gradient Penalty (WGAN-GP) objective on a combination of MAESTRO and GIANT MIDI dataset and evaluated using four objective metrics: Empty Bar Rate (EBR), Average Polyphony, and Rhythmic Pattern Score (RPS) and Pitch Class Histogram Similarity (PCHS). Github link:- AD-GAN GitHub Repository

1 INTRODUCTION

Music generation with the help of deep learning has been a long standing challenge. To capture musical information, previous models including mine have been using piano rolls (MIDI sequences) which offer discrete representation that is suitable for generative modelling. They fundamentally contain a 2-dimensional structure: **two-dimensional structure**: a *temporal* axis governing rhythm and dynamics and a *pitch* axis governing harmony and chord structure. Information like phrasing and tonal structure is governed by the joint distribution of both temporal and pitch axis. Existing convolutional generators either conflate these dimensions via generic 2D convolutions or treat them in strict sequential isolation.

The MidiNet architecture (5) applies standard 2D strided transposed convolutions that expand both the time and pitch axes simultaneously, providing no mechanism to distinguish temporal patterns from harmonic ones.

MuseGAN (4) improves upon this by strictly decoupling temporal and pitch expansion into sequential phases, first growing the time axis to full resolution, then expanding pitch, but this axis-separation means it cannot capture pitch evolution over time.

I propose a third paradigm: **parallel axis decomposition**. At every generator and discriminator stage, a *temporal branch* applies a (5×1) Conv2d kernel that operates strictly along the time dimension, while a *pitch branch* applies five parallel, unlike MuseGAN which operates sequentially, dilated (1×3) Conv2d kernels at musically meaningful dilation rates ($d \in \{1, 3, 4, 7, 12\}$, corresponding to semitone, minor third, major third, perfect fifth, and octave intervals). Both branches operate on the same 4D feature map (B, C, T, P) , and their outputs are fused by a learned 1×1 convolution (MergeBlock).

2 RELATED WORK

2.1 GENERATIVE ADVERSARIAL NETWORKS FOR MUSIC

The foundational GAN framework (1) trains a generator G and discriminator D through a minimax game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (1)$$

where $G : \mathbb{R}^{d_z} \rightarrow \mathbb{R}^{T \times P}$ maps a latent vector z to a piano roll, and $D : \mathbb{R}^{T \times P} \rightarrow [0, 1]$ estimates the probability that its input is real. Prior to MIDINET-based approaches, symbolic music generation relied primarily on recurrent architectures such as DeepBach (2) and C-RNN-GAN (3), which modelled note sequences autoregressively. While effective for monophonic or chorale-style music, these methods struggle with the high-dimensional polyphonic structure of piano music, where multiple notes sound simultaneously across a wide pitch range.

2.2 MIDINET

Midinet, proposed by Yang et al. (5), is one of the earliest convolutional GAN architectures for symbolic music generation. The core operation is the 2D transposed convolution (deconvolution), which maps a low-resolution feature map to a higher-resolution output. For a transposed convolution with kernel $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$, stride (s_h, s_w) , and input $h^{(l)} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$:

$$h^{(l+1)} = \sigma \left(\text{BN} \left(W \star_{\text{T}} h^{(l)} \right) \right), \quad h^{(l+1)} \in \mathbb{R}^{C_{\text{out}} \times s_h H \times s_w W} \quad (2)$$

where $\star_{\text{T}}$ denotes the transposed convolution operator, BN is Batch Normalization, and σ is ReLU (intermediate layers) or Tanh (output layer).

The MidiNet generator uses **joint 2D expansion**: both the time and pitch axes are expanded simultaneously through stride-2 transposed convolutions. Starting from a projected seed $h^{(0)} \in \mathbb{R}^{256 \times 1 \times 11}$:

$$h^{(1)} = \sigma(\text{BN}(\text{ConvT2d}_{s=(2,2)}(h^{(0)}))) \quad \in \mathbb{R}^{128 \times 4 \times 22} \quad (3)$$

$$h^{(2)} = \sigma(\text{BN}(\text{ConvT2d}_{s=(2,2)}(h^{(1)}))) \quad \in \mathbb{R}^{64 \times 8 \times 44} \quad (4)$$

$$h^{(3)} = \sigma(\text{BN}(\text{ConvT2d}_{s=(2,2)}(h^{(2)}))) \quad \in \mathbb{R}^{32 \times 16 \times 88} \quad (5)$$

$$h^{(4)} = \sigma(\text{BN}(\text{ConvT2d}_{s=(2,1)}(h^{(3)}))) \quad \in \mathbb{R}^{16 \times 32 \times 88} \quad (6)$$

$$h^{(5)} = \sigma(\text{BN}(\text{ConvT2d}_{s=(2,1)}(h^{(4)}))) \quad \in \mathbb{R}^{8 \times 64 \times 88} \quad (7)$$

$$\hat{x} = \tanh(\text{ConvT2d}_{s=(2,1)}(h^{(5)})) \quad \in \mathbb{R}^{1 \times 128 \times 88} \quad (8)$$

Note that the kernels in layers 1–3 (4×4), each learned filter, **mixes temporal and pitch information jointly**. The network has no mechanism to distinguish rhythmic patterns (which live along T) from harmonic patterns (which live along P). Any axis-specific representation must be learned entirely from data.

2.3 MUSEGAN

Dong et al. (4) introduced MuseGAN, a multi-track symbolic music GAN. The single-track variant relevant to this comparison employs sequential decoupling of both axes: the generator first expands the temporal axis to full resolution using only $(k \times 1)$ kernels, and only then expands the pitch axis using $(1 \times k)$ kernels.

Formally, the generator is decomposed into two sequential phases:

$$\textbf{Phase 1 (Temporal):} \quad h^{(0)} \in \mathbb{R}^{512 \times 2 \times 1} \quad (9)$$

$$h^{(l+1)} = \sigma(\text{BN}(\text{ConvT2d}_{k=(2,1), s=(2,1)}(h^{(l)}))) \quad l = 0, \dots, 4 \quad (10)$$

$$h^{(5)} \in \mathbb{R}^{64 \times 128 \times 1} \quad (\text{full temporal resolution, pitch} = 1) \quad (11)$$

$$\textbf{Phase 2 (Pitch): } h^{(6)} = \sigma(\text{BN}(\text{ConvT2d}_{k=(1,11),s=(1,11)}(h^{(5)}))) \in \mathbb{R}^{32 \times 128 \times 11} \quad (12)$$

$$h^{(7)} = \sigma(\text{BN}(\text{ConvT2d}_{k=(1,2),s=(1,2)}(h^{(6)}))) \in \mathbb{R}^{16 \times 128 \times 22} \quad (13)$$

$$h^{(8)} = \sigma(\text{BN}(\text{ConvT2d}_{k=(1,2),s=(1,2)}(h^{(7)}))) \in \mathbb{R}^{8 \times 128 \times 44} \quad (14)$$

$$\hat{x} = \tanh(\text{ConvT2d}_{k=(1,2),s=(1,2)}(h^{(8)})) \in \mathbb{R}^{1 \times 128 \times 88} \quad (15)$$

This design guarantees that temporal features $h^{(5)}$ are *fully committed* before any pitch structure is introduced. While this cleanly separates the two domains, it imposes a strictly feedforward information flow: the pitch-expansion layers in Phase 2 have no access to multi-scale temporal context, as the temporal representation is already compressed into a single spatial dimension by the end of Phase 1.

2.4 WASSERSTEIN GAN WITH GRADIENT PENALTY

The original GAN objective (Eq. 1) suffers from training instability due to vanishing gradients when the discriminator becomes too strong. Arjovsky et al. (7) proposed the Wasserstein GAN (WGAN), which replaces the Jensen–Shannon divergence with the Earth Mover’s (Wasserstein-1) distance:

$$W(p_{\text{data}}, p_g) = \sup_{\|D\|_L \leq 1} \{ \mathbb{E}_{x \sim p_{\text{data}}} [D(x)] - \mathbb{E}_{z \sim p_g} [D(G(z))] \} \quad (16)$$

where the supremum is over all 1-Lipschitz functions D . The critic (discriminator) loss and generator loss become:

$$\mathcal{L}_D = \underbrace{\mathbb{E}_z [D(G(z))]}_{\text{push fake scores down}} - \underbrace{\mathbb{E}_x [D(x)]}_{\text{push real scores up}} \quad (17)$$

$$\mathcal{L}_G = -\mathbb{E}_z [D(G(z))] \quad (18)$$

Gulrajani et al. (6) replaced the original weight-clipping enforcement of the Lipschitz constraint with a differentiable **gradient penalty** (GP) computed on interpolations between real and generated samples:

$$\hat{x} = \alpha \cdot x_{\text{real}} + (1 - \alpha) \cdot G(z), \quad \alpha \sim \text{Uniform}(0, 1) \quad (19)$$

$$\text{GP} = \lambda \cdot \mathbb{E}_{\hat{x}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right] \quad (20)$$

The total discriminator loss is therefore:

$$\mathcal{L}_D^{\text{total}} = \mathcal{L}_D + \text{GP} \quad (21)$$

Note that, as the gradient penalty is computed on **individual** interpolated samples $\hat{x}$, the discriminator must not use Batch Normalization, because the running statistics of Batch Norm will mix information across different samples in the batch, which breaks the assumption that gradients are computed independently for each sample. All discriminators in this architecture therefore use Layer Normalization, implemented as $\text{BatchNorm}(1, C)$, which normalizes each sample independently.

2.5 SEPARABLE CONVOLUTIONS

The mathematical equivalence between a 1D convolution along a single axis and a 2D convolution with a degenerate kernel is fundamental to our approach. Consider a 4D feature map $X \in \mathbb{R}^{B \times C \times T \times P}$. A Conv1d with kernel $w \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k}$ applied along the time axis (after folding P into the batch) computes:

$$Y_{1D}[b, c_o, t, p] = \sum_{c_i} \sum_{\tau=0}^{k-1} w[c_o, c_i, \tau] \cdot X[b, c_i, t + \tau, p] \quad (22)$$

A Conv2d with kernel $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times 1}$ applied directly computes:

$$Y_{2D}[b, c_o, t, p] = \sum_{c_i} \sum_{\tau=0}^{k-1} W[c_o, c_i, \tau, 0] \cdot X[b, c_i, t + \tau, p] \quad (23)$$

Setting $W[c_o, c_i, \tau, 0] = w[c_o, c_i, \tau]$ yields $Y_{1D} \equiv Y_{2D}$:

$$\text{Conv1d}(k) \text{ along } T \equiv \text{Conv2d}((k, 1)) \quad (24)$$

The analogous identity holds for the pitch axis: $\text{Conv1d}(k) \text{ along } P \equiv \text{Conv2d}((1, k))$. We exploit this equivalence to eliminate the computationally expensive reshape–permute–convolve–reshape pattern, allowing CuDNN to execute each axis-specific convolution as a single fused kernel over the full (B, C, T, P) tensor.

3 METHODOLOGY

3.1 DATA REPRESENTATION AND DATASET

All models operate on **piano rolls** with shape $(B, 1, T, P)$ where $T = 64$ time steps and $P = 88$ pitch bins (corresponding to the 88 keys of a standard piano, A0–C8). Values are normalized to $[-1, 1]$ for the generator output and rounded to whether the note is present or absent at threshold 0.5 after rescaling to $[0, 1]$ for evaluation.

I have used a combination of **MAESTRO v3.0.0** dataset (8) and 520 specific samples from **GIANT MIDI** dataset, comprising over 300 hours of professional piano performances with aligned MIDI and audio. I combined both the datasets to introduce new variations of pieces apart from the virtuosic pieces present in MAESTRO dataset. Piano rolls are extracted at 16th-note resolution and segmented into non-overlapping 64-timestep windows, yielding approximately **87,000 training samples** and **8564 validation samples**.

3.2 AXIS-DECOMPOSED PIANO GAN ARCHITECTURE

3.2.1 GENERATOR

The generator maps a latent vector $z \sim \mathcal{N}(0, I) \in \mathbb{R}^{64}$ to a piano roll $\hat{x} \in [-1, 1]^{1 \times 64 \times 88}$ through four stages:

Stage 0: Latent Projection

$$z \xrightarrow{\text{Linear}} \mathbb{R}^{48 \times 8 \times 88} \xrightarrow{\text{Reshape}} (B, 48, 8, 88) \quad (25)$$

Since convolutional layers cannot operate on a flat sequence, they require a tensor with time and pitch axes. The reshape step reorganizes this vector into a 4D feature map so that the model can start building patterns across time and pitch.

This step is also what lets the generator actually use the input noise z . If the generator does not depend on z , then every input would lead to the same output, meaning the model would always generate the same piano roll regardless of the noise.

The channel sizes $[48, 32, 24, 12]$ are chosen to support the parallel pitch branches. In each Pitch-Block, the channels are split across five branches, each responsible for a different musical interval. To ensure that each branch has enough capacity to learn meaningful patterns, we require at least two channels per branch. This means the number of input channels must be minimum 10. By using a smaller number of channels like $[32, 24, 16, 8]$, the final stage would have only 8 channels, forcing some branches to operate with just one channel, which is too limited to capture harmonic structure. On the other hand, using a larger configuration like $[64, 48, 32, 12]$ would increase the number of parameters in the initial projection layer.

Stages 1-3: Upsample Blocks

Each UpsampleBlock doubles the time dimension via nearest-neighbour upsampling, then applies the decomposed branches in parallel:

$$x_{\text{up}} = \text{Upsample}_{\times 2}^T(x) \quad (B, c_{\text{in}}, 2T, P) \quad (26)$$

$$y_t = \text{TemporalBlock}(x_{\text{up}}) \quad (B, c_{\text{out}}, 2T, P) \quad [\text{Conv2d}(5, 1)] \quad (27)$$

$$y_p = \text{PitchBlock}(x) \quad (B, c_{\text{out}}, T, P) \quad [5 \times \text{dilated Conv2d}(1, 3)] \quad (28)$$

$$y_p = \text{Upsample}_{\times 2}^T(y_p) \quad (B, c_{\text{out}}, 2T, P) \quad (29)$$

$$\text{out} = \text{MergeBlock}([y_t; y_p]) \quad (B, c_{\text{out}}, 2T, P) \quad [\text{Conv2d}(1, 1)] \quad (30)$$

Checkerboard artifact (10) is a known problem while upsampling using a transposed convolution. Transposed convolutions create uneven kernel overlap which can give more weight to one location and less to other. The previous architectures such as MidiNet and MuseGAN suffer and state this problem as well. In our context, this can create rhythmic noise, I solve this by using nearest-neighbour upsampling which duplicates values uniformly along the temporal axis. Concretely, for an input sequence

$$x = [x_1, x_2, x_3],$$

upsampling by a factor of 2 produces

$$x_{\text{up}} = [x_1, x_1, x_2, x_2, x_3, x_3].$$

In the tensor setting,

$$x \in \mathbb{R}^{(B,C,T,P)} \rightarrow x_{\text{up}} \in \mathbb{R}^{(B,C,2T,P)},$$

where each timestep is duplicated exactly once along the temporal dimension:

$$x[:, :, t, p] \rightarrow x[:, :, 2t, p], \quad x[:, :, 2t+1, p].$$

This ensures that every temporal position contributes equally to the upsampled representation, avoiding the uneven influence seen in transposed convolutions. As a result, the temporal structure is preserved without introducing artificial periodic patterns. Doubling the temporal resolution of the feature maps: ($T = 8 \rightarrow 16 \rightarrow 32 \rightarrow 64$) as the channels reduce to: $[48, 32, 24, 12]$, with the output at 12 channels and $T = 64$.

Stage 4: Enhanced Fusion Block

Three parallel asymmetric 2D convolutions with kernels (3×3) , (3×7) , and (7×3) capture local voice leading, harmonic intervals (the perfect fifth spans 7 semitones), and phrase-level rhythmic patterns respectively. Their outputs are concatenated along the channel dimension to produce a multi-scale feature map $\mathbf{U} \in \mathbb{R}^{B \times 3C \times T \times P}$, which jointly encodes temporal and harmonic structure.

This concatenated representation is then passed through a Squeeze-and-Excitation (SE) module (11), which adaptively calculates the weight of each channel before the final projection. The intuition is simple: not all $3C$ feature channels are equally useful. Some channels may have learned to detect a musically meaningful pattern (e.g., a descending perfect fifth), while others may encode noise or redundant information. The SE module allows the network to give more weights towards the useful channels and suppress the irrelevant ones.

This is achieved in three steps:

Step 1: Squeeze (Global Average Pooling)

Each channel c is compressed into a single scalar by averaging its activations across the entire time-pitch plane:

$$s_c = \frac{1}{T \cdot P} \sum_{t=1}^T \sum_{p=1}^P U_c(t, p), \quad c = 1, \dots, 3C \quad (31)$$

Where $T(\text{No. of time steps}) = 64$ and $P(\text{No. of pitches}) = 88$ in my case. The result $\mathbf{s} \in \mathbb{R}^{3C}$ is a global vector that answers: “how active was each channel on average, over this entire piano roll?”

Step 2: Excitation (Gating)

The pooled vector $\mathbf{s} \in \mathbb{R}^{3C}$ is passed through a two-layer MLP to obtain channel-wise weights:

$$\mathbf{a} = \sigma(W_2 \delta(W_1 \mathbf{s})), \quad (32)$$

where $W_1 \in \mathbb{R}^{(3C/r) \times 3C}$ reduces dimensionality, $W_2 \in \mathbb{R}^{3C \times (3C/r)}$ restores it, δ is ReLU, and σ is the Sigmoid. Each $a_c \in (0, 1)$ acts as a channel importance weight.

Step 3: Reweighting

The feature map $\mathbf{U}$ is rescaled channel-wise:

$$\tilde{U}_c(t, p) = a_c \cdot U_c(t, p). \quad (33)$$

The reweighted tensor $\tilde{\mathbf{U}} \in \mathbb{R}^{B \times 3C \times T \times P}$ is then projected via a 1×1 convolution and tanh to produce the output

$$\hat{x} \in [-1, 1]^{B \times 1 \times T \times P}. \quad (34)$$

3.2.2 DISCRIMINATOR

The discriminator mirrors the generator with DownsampleBlocks that halve the time dimension at each stage via stride-2 (5×1) Conv2d kernels (temporal branch) and avg_pool2d((2, 1)) (pitch branch). Layer Normalization is used throughout to preserve sample independence for the WGAN-GP gradient penalty.

The channel schedule is [12, 16, 24, 48], followed by Global Average Pooling and a linear projection to a scalar critic score.

3.2.3 DILATED PITCH BRANCH: MUSICAL MOTIVATION

The five dilation rates in the PitchBlock are chosen to correspond to intervals with explicit harmonic meaning in Western tonal music:

Dilation d	Semitones	Musical Interval
1	1	Minor 2nd (chromatic / voice leading)
3	3	Minor 3rd
4	4	Major 3rd
7	7	Perfect 5th (most stable interval)
12	12	Octave

Table 1: Musically-motivated dilation rates in the PitchBlock. Each dilation directly encodes a fundamental harmonic interval into the receptive field of the convolutional kernel.

This allows the model to explicitly capture tonal structures; triads, seventh chords, octave doublings without requiring the network to learn these inter-pitch relationships implicitly from data alone.

3.3 BASELINE MODELS

Both baselines are reimplemented to match the mathematical operations described in the original papers, adapted to produce the same output shape (1, 64, 88) and trained under identical conditions.

Component	AD-GAN	MidiNet	MuseGAN
Generator type	Parallel decomposed	Joint 2D	Sequential decoupled
Temporal kernel	Conv2d(5, 1)	Conv2d(4, 4)	ConvT2d(2, 1)
Pitch kernel	$5 \times$ dilated Conv2d(1, 3)	Conv2d(4, 4)	ConvT2d(1, k)
Branch fusion	Learned 1×1 MergeBlock	N/A (joint)	N/A (sequential)
Output head	EnhancedFusionBlock + SE	Tanh	Tanh
Normalization (G)	BatchNorm2d	BatchNorm2d	BatchNorm2d
Normalization (D)	BatchNorm(1, C)	None	None
G parameters	[2.2M]	[3.6M]	[1.5M]
D parameters	[27k]	[8.5M]	[1.6M]

Table 2: Architectural comparison of the three models. All models output ($B, 1, 64, 88$) piano rolls. AD-GAN is highly parameter-efficient, whereas MidiNet’s parameter count scales quadratically due to its reliance on standard 2D convolutions.

All three models are trained using the WGAN-GP objective:

$$\mathcal{L}_D = \mathbb{E}_z[D(G(z))] - \mathbb{E}_x[D(x)] + \lambda \cdot \text{GP} \quad (35)$$

$$\mathcal{L}_G = -\mathbb{E}_z[D(G(z))] \quad (36)$$

Hardware: Kaggle T4 $\times$ 2 (2 $\times$ NVIDIA Tesla T4, 16GB VRAM)

Hyperparameter	Value
Latent dimension z	64
Batch size	32
Critic steps n_{critic}	3
Learning rate (G and D)	1×10^{-4}
Adam β_1 / β_2	0.5 / 0.9
GP coefficient λ	10.0
Training steps	25,000
Checkpoint frequency	Every 500 steps

Table 3: Training hyperparameters (identical across all three models). Since the models crashed and could not complete total steps, the maximum training step size shown in this paper is 6,000

4 EVALUATION METRICS

I evaluate all models using three objective metrics computed on binary piano rolls thresholded at 0.5:

4.1 EMPTY BAR RATE (EBR)

The fraction of generated windows in which no note is active at any time step. $\text{EBR} = 0$ indicates no silent bars (ideal); $\text{EBR} = 1$ indicates complete mode collapse.

$$\text{EBR} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[\sum_{t,p} \hat{x}_i[t,p] = 0 \right] \quad (37)$$

4.2 AVERAGE POLYPHONY

The mean number of simultaneously active pitches per time step, averaged only over non-silent steps. Real piano music typically exhibits polyphony in the range of 2–6 simultaneous notes.

$$\text{Poly} = \frac{1}{|\{t : a(t) > 0\}|} \sum_{t:a(t)>0} \sum_p \hat{x}[t,p], \quad a(t) = \sum_p \hat{x}[t,p] \quad (38)$$

4.3 RHYTHMIC PATTERN SCORE (RPS)

The average normalized autocorrelation of the onset density signal at lags 4, 8, and 16 time steps (corresponding to 1, 2, and 4 beats in a 16th-note grid). Higher RPS indicates more regular, metrically coherent rhythm.

$$\text{RPS} = \frac{1}{3} \sum_{\ell \in \{4,8,16\}} \frac{\sum_t d(t) \cdot d(t+\ell)}{\sqrt{\sum_t d(t)^2 \cdot \sum_t d(t+\ell)^2} + \epsilon} \quad (39)$$

where $d(t) = \sum_p \mathbf{1}[\hat{x}[t,p] > 0 \wedge \hat{x}[t-1,p] = 0]$ is the onset density at time step t . The intuitive meaning behind $d(t)$ is 'how many notes start at time t '. If the condition in $d(t)$ satisfies then 1 else 0 hence it sums over all the ones.

4.4 PITCH CLASS HISTOGRAM SIMILARITY (PCHS)

The chi-square distance between 12-bin pitch-class histograms of real and generated samples. Each of the 88 piano keys is mapped to one of 12 pitch classes (C, C#, ..., B), and the note activations are aggregated and normalized. Lower PCHS indicates that the generated music has a tonal distribution closer to real piano music.

$$\text{PCHS} = \sum_{c=0}^{11} \frac{(h_{\text{real}}(c) - h_{\text{gen}}(c))^2}{h_{\text{real}}(c) + h_{\text{gen}}(c) + \epsilon} \quad (40)$$

where $h(c) = \frac{1}{Z} \sum_{i: \text{pc}(i)=c} \sum_t \hat{x}[t, i]$ is the normalized pitch-class histogram and $\text{pc}(i) = (i + 9) \bmod 12$ maps piano key index i ($A0 = 0$) to its pitch class.

5 RESULTS & ANALYSIS

5.1 EXPERIMENTAL CONSTRAINTS

Several challenges arose that shaped the outcome and interpretation of these results.

Sequence Length Reduction

The original implementation targeted 8-bar windows ($T = 128$ time steps), which was the natural segmentation of the MAESTRO dataset at 16th-note resolution. However, the $2 \times \text{T4}$ environment could not sustain the corresponding VRAM requirements at a batch size large enough for stable WGAN-GP training. The window size was therefore reduced to **4 bars** ($T = 64$), halving the sequence length and making the task tractable at batch size 32 with `n_critic = 3`. The trade off: shortening the window enabled the model to train but limited model’s exposure to long ranged phrases.

Training steps

All reported metrics are from preliminary checkpoints that were saved before premature termination of all 4 runs.

Model	Steps Completed	Epochs
MidiNet	6,000	5.50
MuseGAN	4,700	4.28
Baseline 2D	4,900	4.46
AD-GAN	1,400	1.27

Table 4: Training steps and epochs completed before Kaggle container termination.

5.2 QUANTITATIVE RESULTS

Table 5 reports the four evaluation metrics for all models at their respective early checkpoints, computed over $N = 64$ generated samples with a fixed random seed.

Model	EBR ↓	Polyphony	RPS ↑	PCHS ↓
MidiNet (step 6000)	0.0000	2.579	0.3889	0.0158
MuseGAN (step 4500)	1.0000	0.000	0.0000	1.0000
Baseline 2D (step 4500)	0.0000	2.757	0.4472	0.1782
AD-GAN (step 1000)	0.0000	5.193	0.3776	0.2684

Table 5: Objective evaluation of preliminary checkpoints ($N = 64$ generated samples, fixed seed). EBR: lower is better (0 = no silent bars). RPS: higher is better. PCHS: lower is better (closer to real tonal distribution). Polyphony target range: 2–6 for piano.

5.3 ANALYSIS OF MODEL PERFORMANCE

MidiNet and Baseline-2D achieve the best overall quality. MidiNet reaches 6,000 steps, the longest training of all models and produces better results on polyphony (2.579) and PCHS of only 0.0158 (near-perfect tonal alignment with real piano music). Informal listening evaluation of the generated MIDI files confirms this: MidiNet samples contain recognisable rhythmic patterns and plausible harmonic progressions, making them the most musically convincing outputs of this study. The Baseline 2D model achieves similar structural metrics at fewer steps, confirming that nearest-neighbour upsampling introduces a small inductive bias when training compute is scarce.

Why MidiNet trains faster despite more parameters. MidiNet has 3.66M generator parameters and 8.53M discriminator parameters, compared to AD-GAN’s 2.24M and 27K respectively. Despite the much larger parameter count, MidiNet completes each training step faster as the WGAN-GP gradient penalty in AD-GAN must backpropagate through the deeper layers in AD-GAN, which is computationally more expensive per critic step than backpropagating through MidiNet’s linear stack of transposed convolutions. AD-GAN, by contrast, executes multiple sequential operations per block: a TemporalBlock, a PitchBlock (containing five parallel dilated convolutions), a MergeBlock, and an upsample, creating a longer computation graph with more synchronization points on the GPU. The net result is that MidiNet achieves roughly $6\times$ more training steps within the same Kaggle session budget.

AD-GAN at 1,000 steps. AD-GAN shows a promising RPS of 0.378, which is competitive with MidiNet at six times fewer training steps. Its polyphony of 5.193, while technically within the target range, leans towards the upper bound, suggesting that the generator is still producing an excessive number of simultaneous notes and has not learnt to apply sparsity. The PCHS of 0.2684, while higher than MidiNet’s 0.0158, AD-GAN’s pitch branch explicitly encodes harmonic intervals (minor third, major third, perfect fifth, octave), I expect the PCHS to improve substantially with additional training.

MuseGAN fails to produce output. MuseGAN exhibits complete mode collapse (EBR = 1.000, Polyphony = 0.000) at its 4,500-step checkpoint. The generator converges to the trivial solution of producing a zero-density output that the discriminator cannot distinguish from silence. The MuseGAN architecture’s sequential temporal-then-pitch design imposes a two-phase training dynamic: the temporal branch must first develop a stable representation before pitch expansion becomes meaningful. At only 4,500 steps, the temporal branch had not yet learnt a coherent rhythm, causing the pitch expansion phase to collapse. Longer training, would be required before MuseGAN produces structured output.

5.4 QUANTITATIVE METRICS COMPARISON

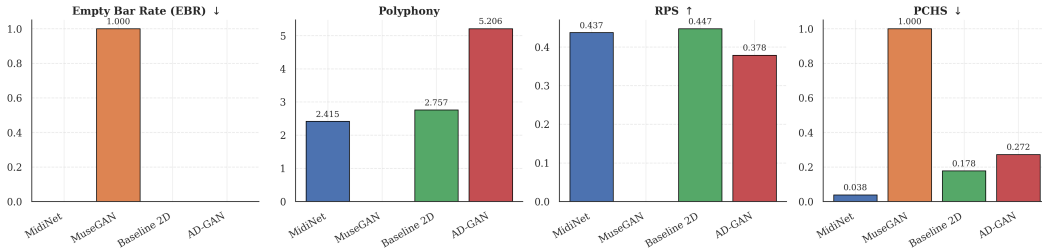


Figure 1: Comparison of objective evaluation metrics across all four preliminary checkpoints. MuseGAN’s complete mode collapse (EBR = 1.0) is clearly visible. AD-GAN, despite its shorter training, is comparable to MidiNet on RPS.

5.5 QUALITATIVE ANALYSIS — GENERATED PIANO ROLLS

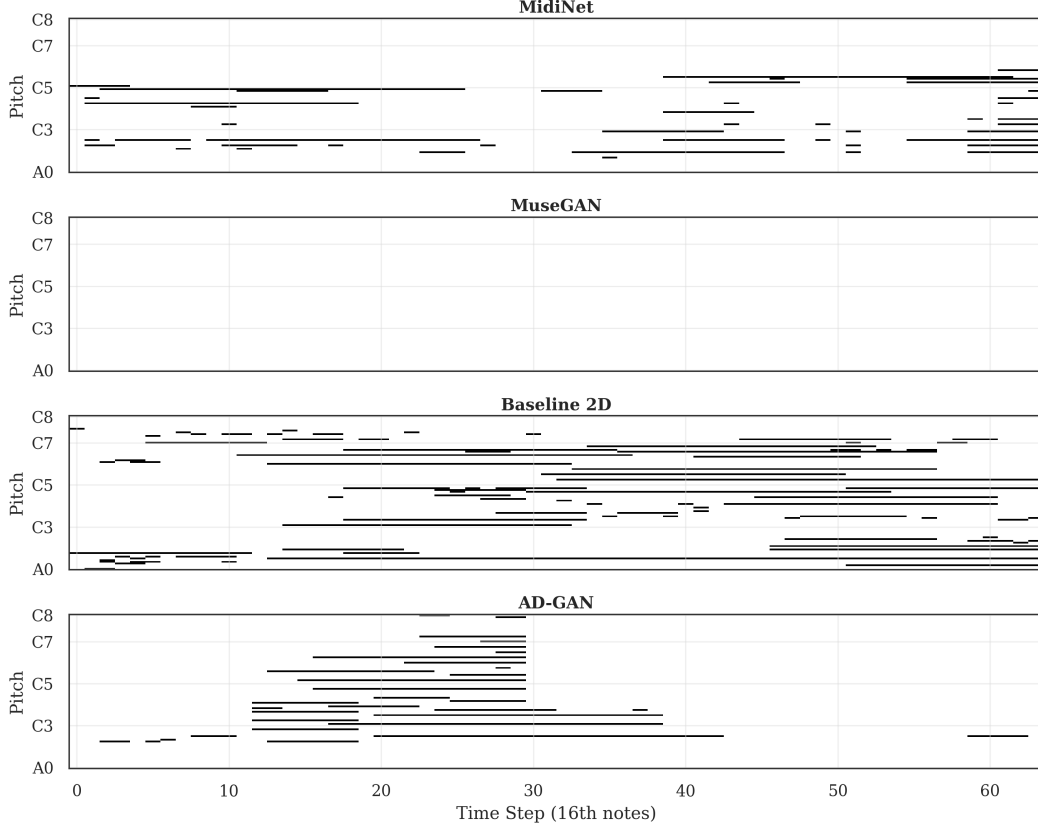


Figure 2: Representative generated piano rolls from early checkpoints. Top to bottom: MidiNet (step 6000), MuseGAN (step 4500, fully collapsed to silence), Baseline 2D (step 4500), AD-GAN (step 1000). Each row spans 64 time steps $\times$ 88 pitch bins. Active notes are shown in black. MidiNet shows clear rhythmic structure with sparse, well-separated note clusters. AD-GAN shows denser chord activations consistent with an early over-activation phase (high polyphony).

6 CONCLUSION

I propose the Axis-Decomposed GAN (AD-GAN), a WGAN-GP-based architecture for symbolic piano music generation that explicitly separates temporal and harmonic processing through parallel separable Conv2d branches with musically-motivated dilation rates. The pitch branch’s dilation rates ($d = 1, 3, 4, 7, 12$) directly encode harmonic intervals of Western classical music into the model’s receptive field.

Preliminary results indicate that at lower computational budgets (e.g., $\leq 4,500$ training steps), standard baseline models like MidiNet and the 2D Baseline out-perform AD-GAN on objective metrics such as Average Polyphony and RPS. This suggests that the parallel axis decomposition requires a longer training period to properly align and fuse the independent temporal and harmonic feature maps.

Future work includes: (1) extending the architecture to multi-instrument tracks using per-track generator networks, (2) incorporating conditioning on musical key signature, tempo, and style, and (3) conducting full-length training runs to evaluate whether AD-GAN surpasses baselines at convergence.

REFERENCES

- [1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [2] Gaëtan Hadjeres, François Pachet, and Frank Nielsen. DeepBach: A steerable model for Bach chorales generation. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2017.
- [3] Olof Mogren. C-RNN-GAN: Training a continuous recurrent neural network with adversarial feedback. *arXiv preprint arXiv:1611.09904*, 2016.
- [4] Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang. MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [5] Li-Chia Yang, Szu-Yu Chou, and Yi-Hsuan Yang. MidiNet: A convolutional generative adversarial network for symbolic-domain music generation. In *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2017.
- [6] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of Wasserstein GANs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [7] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2017.
- [8] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations (ICLR)*, 2019.
- [9] Qiuqiang Kong, Bochen Li, Jitong Chen, and Yuxuan Wang. GiantMIDI-Piano: A large-scale MIDI dataset for classical piano music. *arXiv preprint arXiv:2010.07061*, 2020.
- [10] Augustus Odena, Vincent Dumoulin, and Chris Olah. Deconvolution and checkerboard artifacts. *Distill*, 2016. <https://distill.pub/2016/deconv-checkerboard/>
- [11] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.